AULA 1

ORDENAÇÃO

ÍNDICE

01 Planejamento Semestral

02 Time Limit Exceed

03 Merge Sort

04 Quick Sort

PLANEJAMENTO

- 10 aulas totais;
- Toda segunda-feira, das 12h30 às 14h;
- 45 mins teoria (Peres) + 45 mins prática (Marco);
- Estímulo à leitura do material base (Noic).

DIVISÃO DE CONTEÚDOS

- Aula 1: Ordenação;
- Aula 2: Explorando STL;
- Aula 3: Busca em Grafo;
- Aula 4: Menor Caminho;
- Aula 5: AGM;
- Aula 6: DFS em Grid;
- Aula 7: BFS 01;
- Aula 8: Dijkstra com Vértices Auxiliares;
- Aula 9: Ordenação Topológica;
- Aula 10: Caminho e Ciclo Euleriano.

TIME LIMIT EXCEED

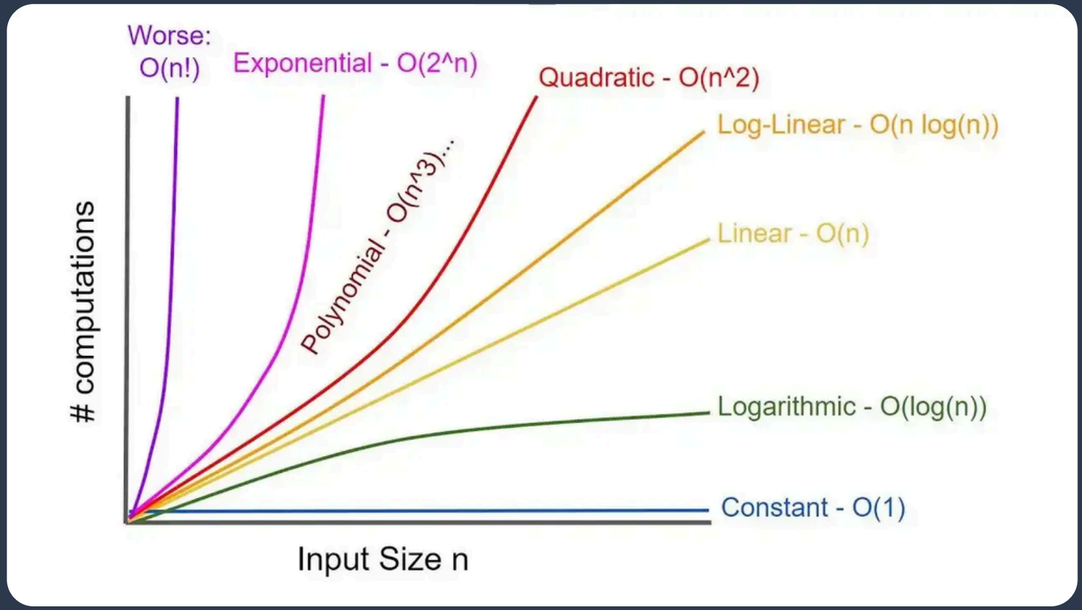
COMPLEXIDADE ASSINTÓTICA

- Mede a eficiência de um algoritmo à medida que o tamanho da entrada (n) cresce para o infinito;
- Foco no comportamento de longo prazo (pior caso) em vez de medições precisas;
- Utiliza notação Big O para classificar algoritmos(ex: linear $O(n)$, quadrática $O(n^2)$.

TIME LIMIT EXCEED

POR QUE $O(n^2)$ NÃO SERVE?

- Em competições como OBI e ICPC, geralmente temos 1 segundo de limite, o que nos permite fazer cerca de 10^8 operações;
- Se n = 10^5, algoritmos $O(n^2)$ fariam 10^10 operações e ganhariam Time Limit Exceeded (TLE);
- Precisamos de algoritmos *O(nlog(n))*!



TIME LIMIT EXCEED

ALGORITMOS O(N^2)

- Bubble Sort;
- Selection Sort;
- **Problema:** Resolvem o problema olhando cada par de elementos.

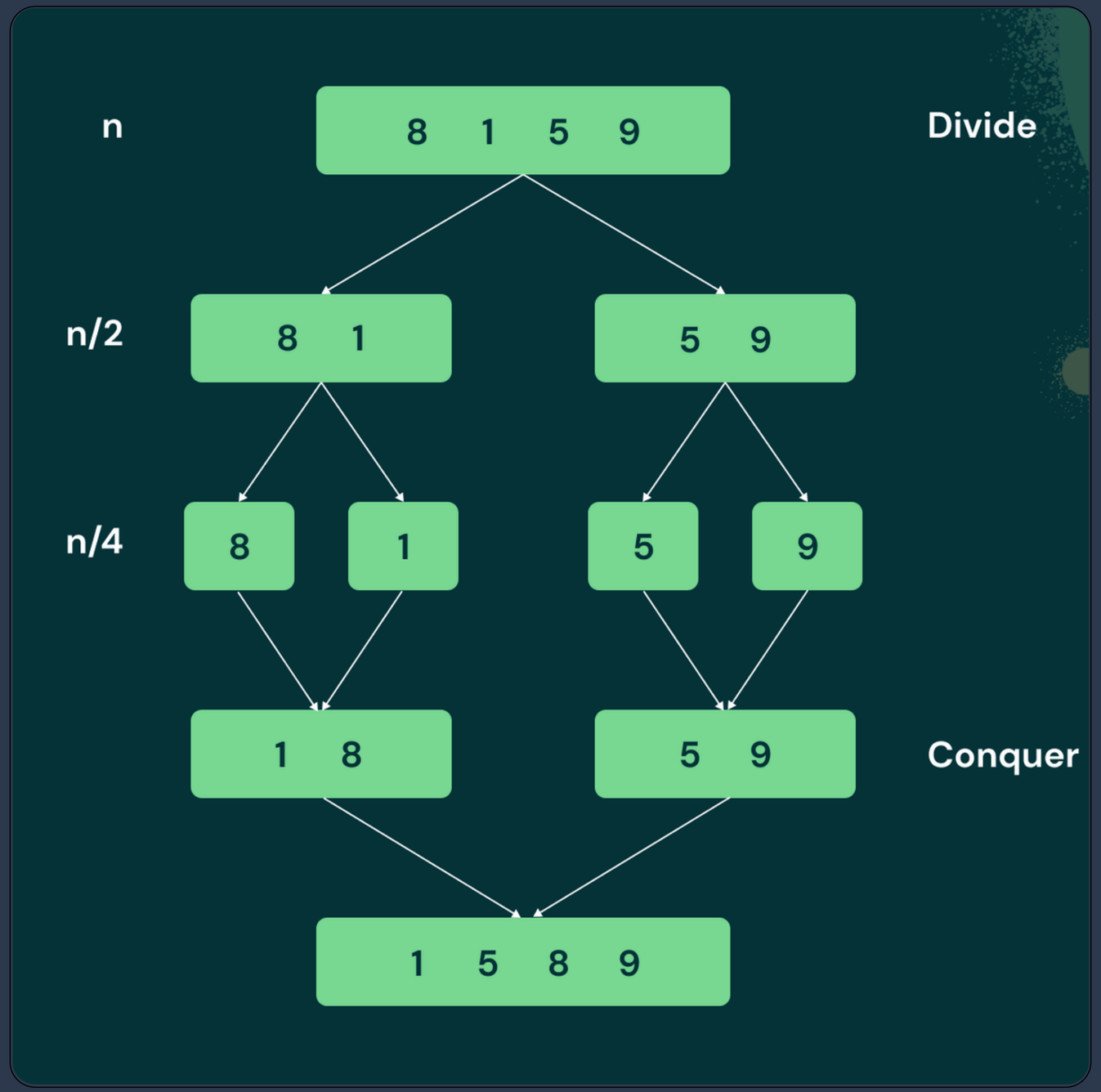
ALGORITMOS O(NLOG(N))

- Merge Sort;
- Quick Sort.

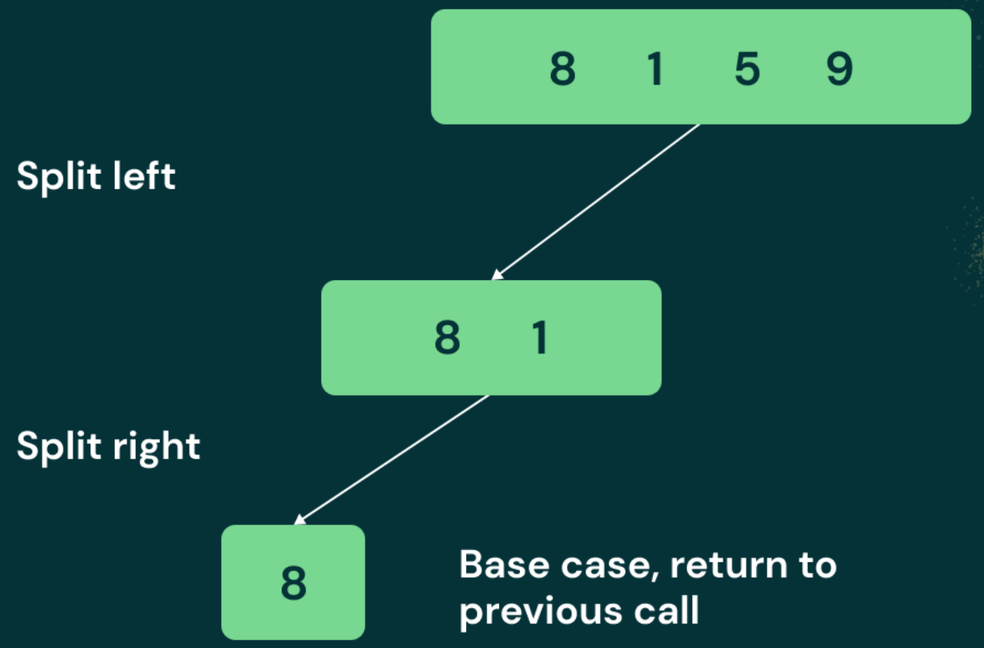
MERGE SORT

DIVIDIR PARA CONQUISTAR

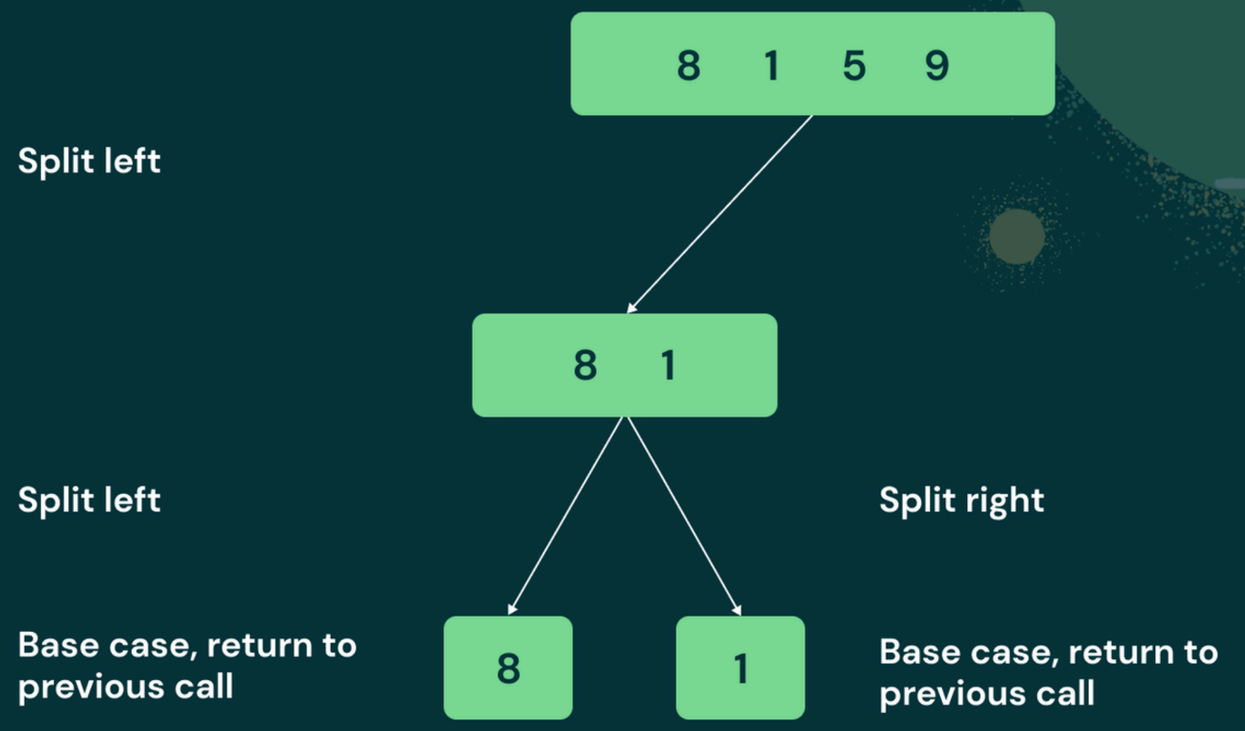
- **Ideia:** É mais fácil juntar dois vetores já ordenados do que ordenar um único grande vetor.
- **O algoritmo:**
  - *Divisão:* Divide o vetor no meio até que sobrem apenas vetores de tamanho 1 (por definição, ordenados);
  - *Conquista:* Junta os vetores menores em ordem.
- O algotitmo é gerenciado por um função recursiva que divide os elementos até o caso base.


n
8 1 5 9
Divide
n/2
8 1
5 9
n/4
8
1
5
9
1 8
5 9
Conquer
1 5 8 9

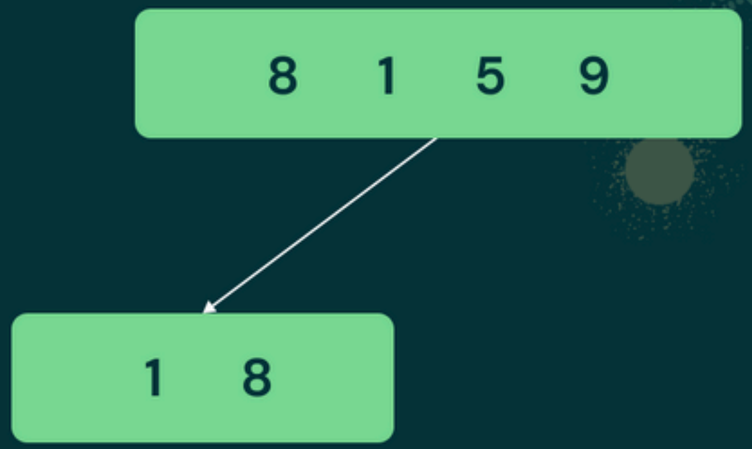
Step 1


8 1 5 9
Split left
8 1
Split right
8
Base case, return to previous call

Step 2


8 1 5 9
Split left
8 1
Split left
Split right
Base case, return to previous call
8
1
Base case, return to previous call

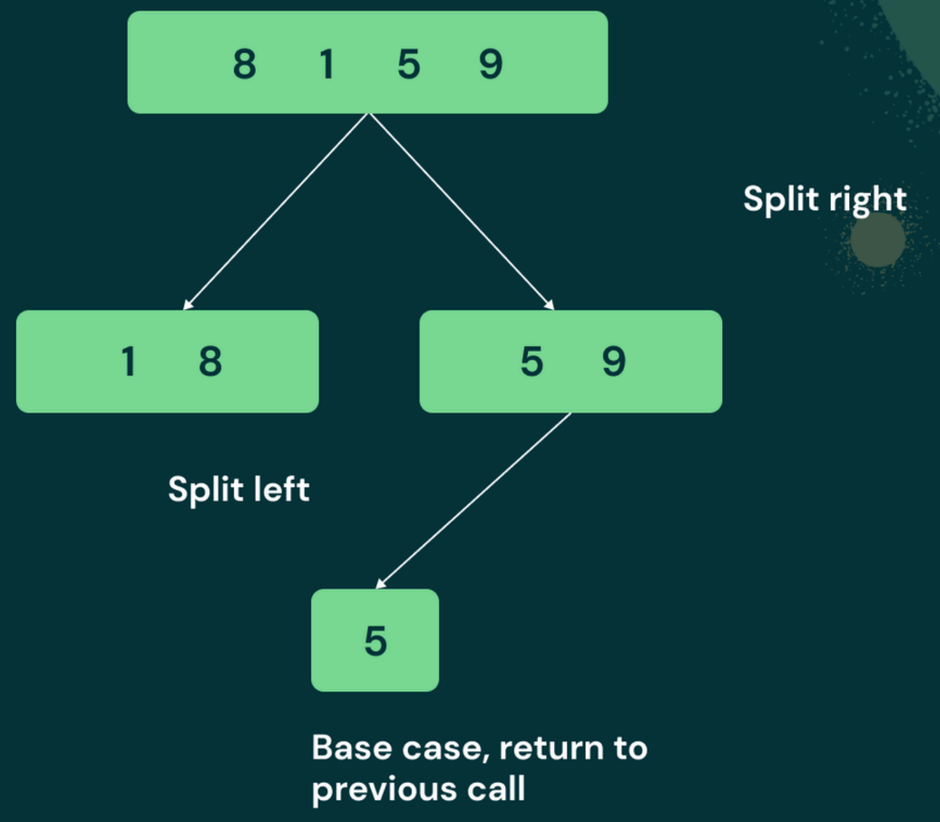
Step 3



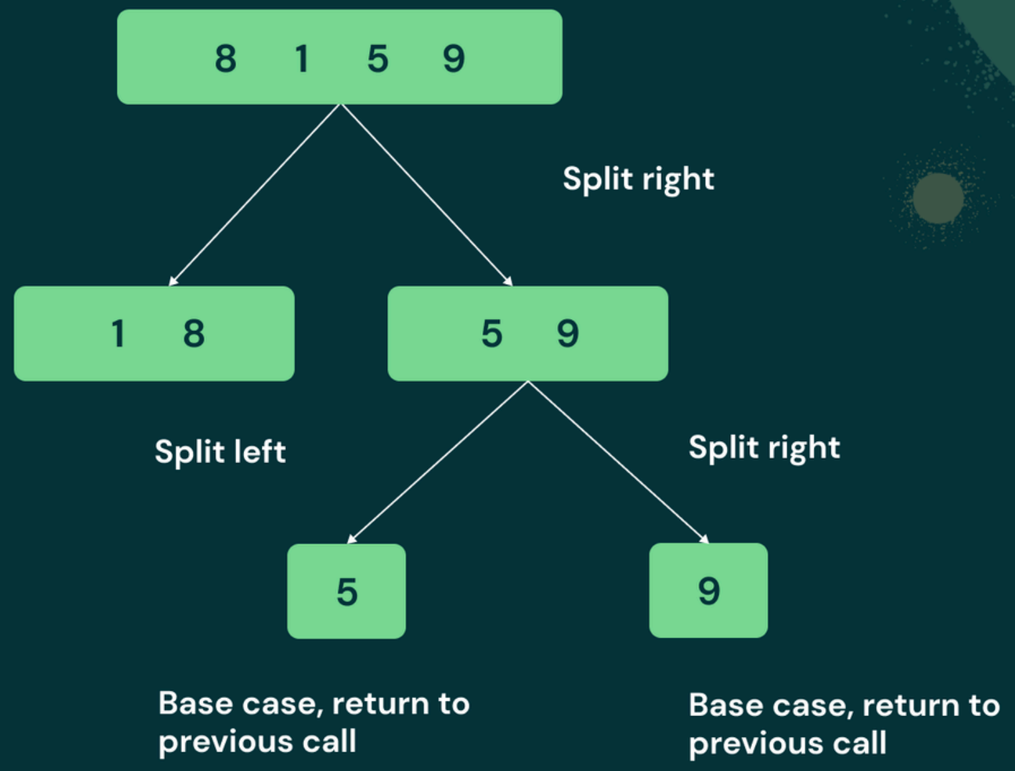
Merge subroutine sorts and merges

Return to previous call

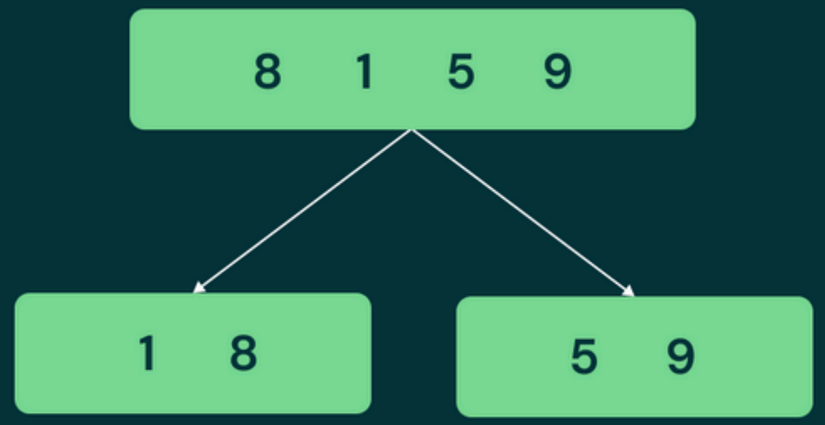
Step 4


8 1 5 9
Split right
1 8
5 9
Split left
5
Base case, return to previous call

Step 5


8 1 5 9
Split right
1 8
5 9
Split left
Split right
5
9
Base case, return to previous call
Base case, return to previous call

Step 6



Merge subroutine sorts and merges

Return to previous call

Step 7

1 5 8 9

Sorted Array

Merge subroutine sorts and merges

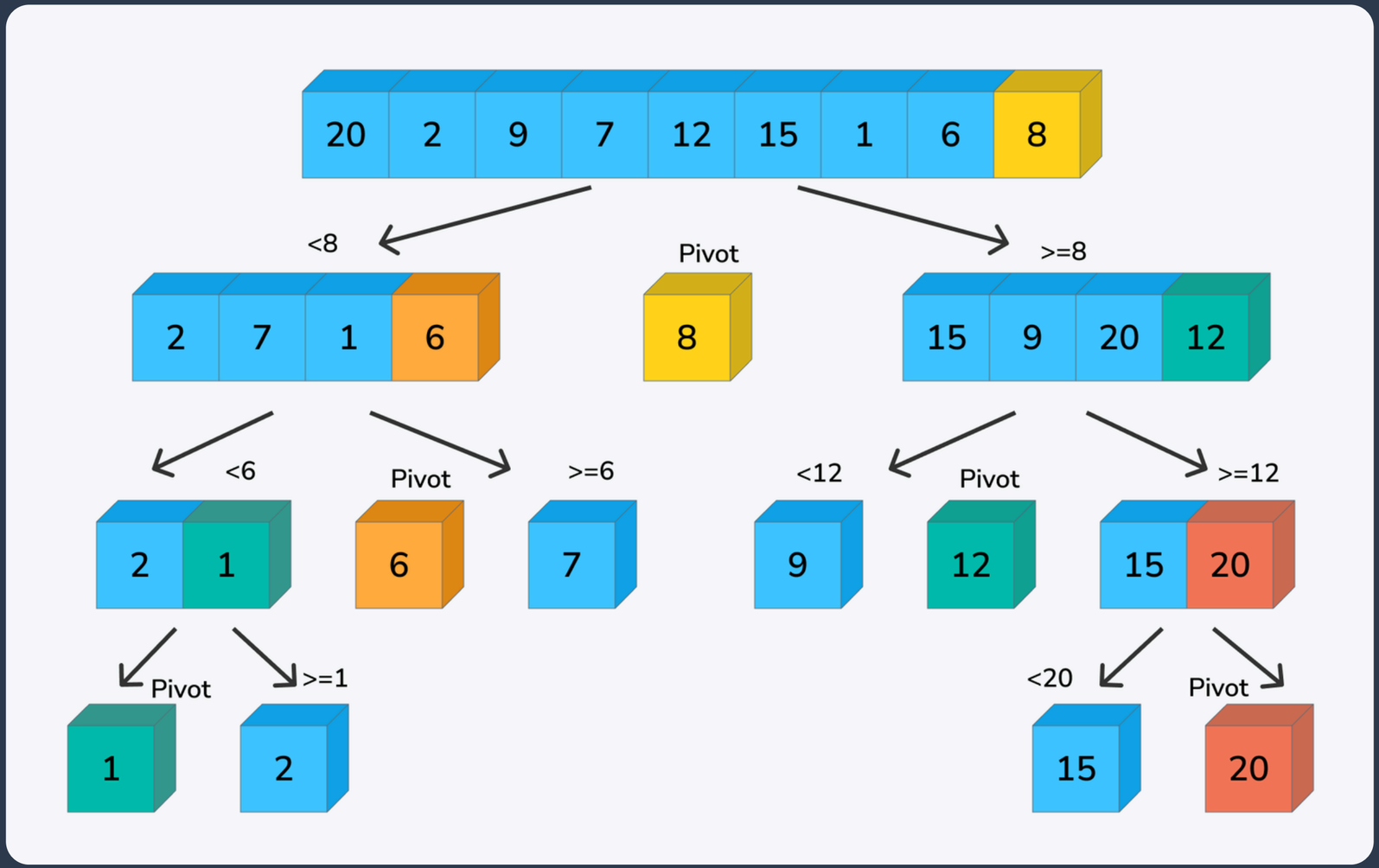
MERGE SORT

CONCLUSÕES

- **Complexidade *O(nlog(n))*:**
  - O vetor é dividido *log(n)* vezes (altura da árvore recursiva);
  - Em cada nível dessa divisão, o processo de merge consome *O(n)* operações.
- **Foco na Maratona:**
  - Merge Sort resolve o problema clássico de "Número de Inversões" (ex: o problema Balé, da OBI);
  - É possível contar quantos elementos menores estão à direita de um elemento simplesmente adaptando uma linha no momento do merge.

QUICK SORT

- **Ideia:** Ao invés de dividir o vetor simplesmente ao meio, escolhemos um elemento chamado Pivô. O algoritmo joga todos os números menores que o pivô para a esquerda e os maiores para a direita (particionamento);
- **Complexidade:** $O(n\log(n))$ no caso médio. Porém, tem o pior caso de $O(n^2)$ se o vetor já estiver ordenado e o pivô for mal escolhido.


20 2 9 7 12 15 1 6 8
<8
Pivot
>=8
2 7 1 6
8
15 9 20 12
<6
Pivot
>=6
<12
Pivot
>=12
2 1
6
7
9
12
15 20
Pivot
>=1
<20
Pivot
1
2
15
20

QUICK SORT

PERSPECTIVA PARA MARATONA

- Raramente implementaremos o Quick Sort do zero;
- Utilizamos a função ***sort*** da biblioteca ***<algorithm>*** do C++;
- O ***std::sort*** usa internamente uma versão extremamente otimizada (como o IntroSort, que mistura Quick Sort, Heap Sort e Insertion Sort) para garantir o *O(nlog(n))* sempre e evitar TLE.

Bibliografia

Curso Noic - Programação Básica: Aula 7
noic.com.br/materiais-informatica/curso/basic-05/

Everything You Need to Know About Merge Sort
crio.do/blog/merge-sort-algorithm-dsa/

The Complete Quick Sort Guide
crio.do/blog/quick-sort/